Practical: Create a Form with JavaScript Validation

Aim: To create an HTML form that validates user input using JavaScript.

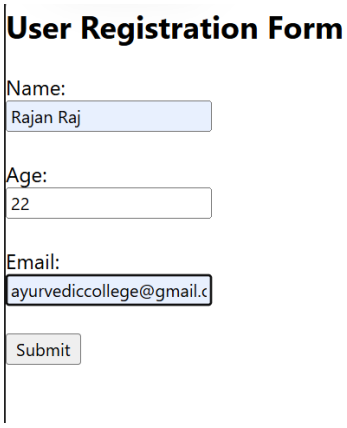
Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Form with JavaScript Validation</title>
    <script>
        function validateForm() {
            // Get values from form
            var name = document.forms["userForm"]["name"].value;
            var age = document.forms["userForm"]["age"].value;
            var email = document.forms["userForm"]["email"].value;

            // Name validation
            if (name == "") {
                alert("Name must be filled out");
                return false;
            }

            // Age validation
            age = Number(age);
            if (isNaN(age) || age < 1 || age > 150) {
                alert("Age must be a number between 1 and 150");
                return false;
            }

            // Email validation (basic)
            var emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
            if (!email.match(emailPattern)) {
                alert("Please enter a valid email address");
                return false;
            }

            alert("Form submitted successfully!");
            return true;
        }
    </script>
</head>
```

```
<body>

<h2>User Registration Form</h2>

<form name="userForm" onsubmit="return validateForm()">
    <label for="name">Name:</label><br>
    <input type="text" id="name" name="name" placeholder="Enter your
name"><br><br>

    <label for="age">Age:</label><br>
    <input type="text" id="age" name="age" placeholder="Enter your
age"><br><br>

    <label for="email">Email:</label><br>
    <input type="text" id="email" name="email" placeholder="Enter
your email"><br><br>

    <input type="submit" value="Submit">
</form>

</body>
</html>
```

Output:

- A form with fields for **Name**, **Age**, and **Email**.
- When submitted:
    - **Name** cannot be empty.
    - **Age** must be a number between **1 and 150**.
    - **Email** must follow a valid email format.
- If all validations pass, an alert **"Form submitted successfully!"** appears.



Observation:

- JavaScript can validate form inputs **before submission**.
- Alerts are displayed if user enters invalid data.
- Prevents incorrect or incomplete form submission.

Conclusion:

- JavaScript validation improves **user experience** by catching errors immediately.
- Helps maintain **data integrity** before sending it to the server.

Practical: Display Information Box on Page Load using JavaScript

Aim: To create a web page that shows an information box (alert) as soon as the page loads.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Information Box on Page Load</title>
    <script>
        // Function to display alert when page loads
        function showInfo() {
            alert("Welcome! This is an information box displayed on
page load.");
        }
    </script>
</head>
<body onload="showInfo()">

<h2>JavaScript Information Box Example</h2>
<p>This page displays an alert box automatically when the page is
loaded.</p>

</body>
</html>
```

JavaScript Information Box Example

This page displays an alert box automatically when the page is loaded.

Output:

- As soon as the page loads, a pop-up alert box appears with the message: **"Welcome! This is an information box displayed on page load."**
- After clicking **OK**, the page content is visible:
  - Heading: *JavaScript Information Box Example*
  - Paragraph: *This page displays an alert box automatically when the page is loaded.*

Observation:

- The `onload` event in the <body> tag executes the JavaScript function as soon as the page is fully loaded.
- The `alert()` function creates a simple information box.

Conclusion:

- JavaScript can trigger actions automatically when the page loads.
- Useful for welcome messages, notifications, or initial instructions for users.

Practical: Change Background Color After 5 Seconds Using JavaScript

Aim: To create a web page that changes its background color automatically 5 seconds after the page loads.

Code:

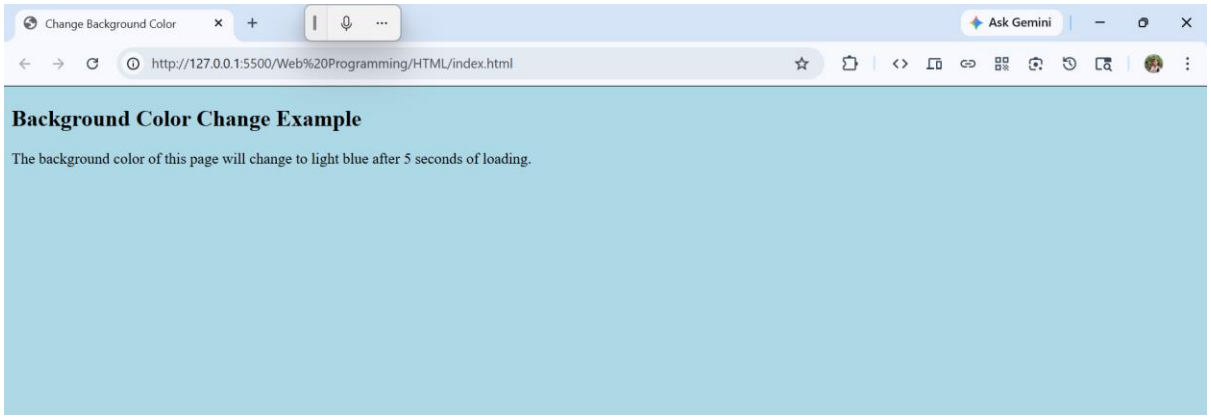
```
<!DOCTYPE html>
<html>
<head>
    <title>Change Background Color</title>
    <script>
        // Function to change background color
        function changeBackground() {
            document.body.style.backgroundColor = "lightblue";
        }

        // Call the function after 5 seconds
        window.onload = function() {
            setTimeout(changeBackground, 5000); // 5000 milliseconds
= 5 seconds
        }
    </script>
</head>
<body>

<h2>Background Color Change Example</h2>
<p>The background color of this page will change to light blue after
5 seconds of loading.</p>

</body>
</html>
```

Output:

- Initially, the page has the default background color (usually white).
- After **5 seconds**, the background color automatically changes to **light blue**.
- Heading: *Background Color Change Example*
- Paragraph: *The background color of this page will change to light blue after 5 seconds of loading.*

Background Color Change Example

The background color of this page will change to light blue after 5 seconds of loading.



Background Color Change Example

The background color of this page will change to light blue after 5 seconds of loading.

Observation:

- `setTimeout()` is used to execute a function after a specified time delay (5000 milliseconds = 5 seconds).
- `document.body.style.backgroundColor` changes the background color dynamically.

Conclusion:

- JavaScript can perform **delayed actions** on a page using `setTimeout()`.
- Useful for effects, notifications, or dynamic styling after a time delay.

Practical: Dynamically Bold, Italic, and Underline Text Using JavaScript

Aim: To create a web page where the user can apply **bold, italic, and underline styles** to text dynamically using buttons.

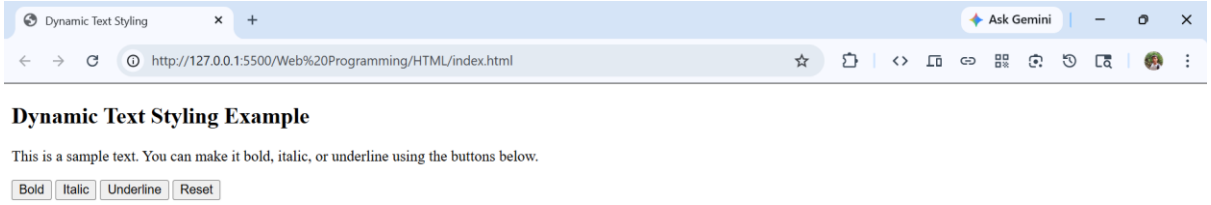
Code:

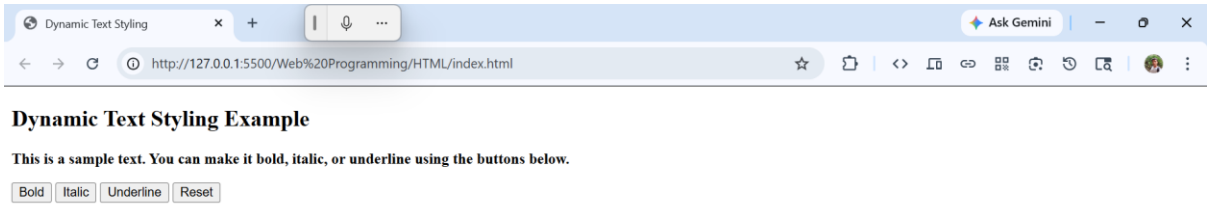
```
<!DOCTYPE html>
<html>
<head>
    <title>Dynamic Text Styling</title>
```

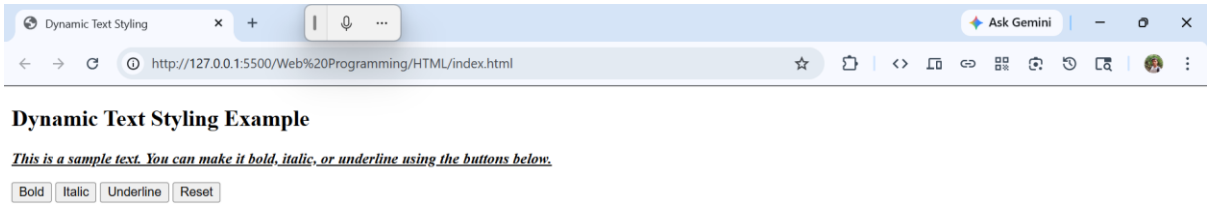
```
    <script>
        // Function to make text bold
        function makeBold() {
            var text = document.getElementById("text");
            text.style.fontWeight = "bold";
        }

        // Function to make text italic
        function makeItalic() {
            var text = document.getElementById("text");
            text.style.fontStyle = "italic";
        }

        // Function to underline text
        function makeUnderline() {
            var text = document.getElementById("text");
            text.style.textDecoration = "underline";
        }

        // Function to reset text style
        function resetStyle() {
            var text = document.getElementById("text");
            text.style.fontWeight = "normal";
            text.style.fontStyle = "normal";
            text.style.textDecoration = "none";
        }
    </script>
</head>
<body>

<h2>Dynamic Text Styling Example</h2>

<p id="text">This is a sample text. You can make it bold, italic, or
underline using the buttons below.</p>

<button onclick="makeBold()">Bold</button>
<button onclick="makeItalic()">Italic</button>
<button onclick="makeUnderline()">Underline</button>
<button onclick="resetStyle()">Reset</button>

</body>
</html>
```

Output:

- Paragraph displays: *"This is a sample text. You can make it bold, italic, or underline using the buttons below."*
- **Buttons:**
  - **Bold:** Makes text bold.
  - **Italic:** Makes text italic.
  - **Underline:** Underlines the text.
  - **Reset:** Returns text to normal style.







Observation:

- JavaScript can dynamically change the **style properties** of HTML elements using `element.style`.
- Users can apply or remove text styles interactively.

Conclusion:

- Dynamic styling improves **user interaction** on web pages.
- Useful in text editors, form formatting, and other interactive applications.

Practical: Display Hidden Div Using JavaScript

Aim: To create a web page where clicking on a player's name reveals a hidden div containing player statistics.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Show Hidden Div Example</title>
    <style>
        /* Initially hide the stats div */
        #playerStats {
            display: none;
            border: 1px solid black;
            padding: 10px;
            width: 300px;
            background-color: #f0f8ff;
            margin-top: 10px;
        }

        /* Style player name as clickable */
        .playerName {
            color: blue;
            cursor: pointer;
            text-decoration: underline;
        }
    </style>
    <script>
        function showStats() {
            var statsDiv = document.getElementById("playerStats");
            if (statsDiv.style.display === "none") {
                statsDiv.style.display = "block";
            } else {
                statsDiv.style.display = "none";
            }
        }
    </script>
</head>
<body>

<h2>Player Information</h2>
```

```
<p>Click on the player’s name to view stats:</p>
<p class="playerName" onclick="showStats()">Lionel Messi</p>

<div id="playerStats">
    <h3>Player Stats</h3>
    <p>Age: 36</p>
    <p>Club: Inter Miami</p>
    <p>Position: Forward</p>
    <p>Goals Scored: 800+</p>
</div>

</body>
</html>
```

Output:

- The player name **“Lionel Messi”** appears as **blue and underlined**.
- Initially, the **player stats div** is hidden.
- When the user clicks on the player name, the **stats div becomes visible**, showing:
    - Age: 36
    - Club: Inter Miami
    - Position: Forward
    - Goals Scored: 800+
- Clicking again **hides the div**.

Player Information

Click on the player's name to view stats:

Lionel Messi

Player Information

Click on the player's name to view stats:

Lionel Messi

Player Stats

Age: 36

Club: Inter Miami

Position: Forward

Goals Scored: 800+

Observation:

- `onclick` event triggers the JavaScript function to toggle the visibility.
- `style.display` is used to show or hide HTML elements dynamically.

Conclusion:

- JavaScript can dynamically display hidden content based on user interaction.
- Useful for interactive dashboards, FAQs, and information panels.

Practical: Prompt User for Name and Display It

Aim: To create a web page that asks the user for their name using a prompt and displays it on the page.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Display User Name</title>
    <script>
        function displayUserName() {
            // Prompt user to enter their name
```

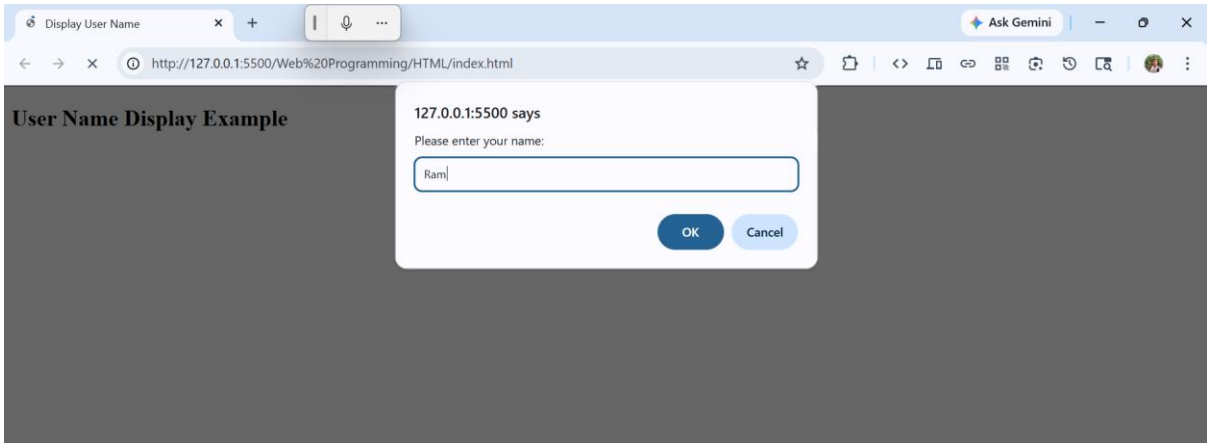
```
            var userName = prompt("Please enter your name:");

            // Display the name on the page
            if (userName != null && userName != "") {
                document.getElementById("greeting").innerHTML =
                   "Hello, " + userName + "! Welcome to our
website.";
            } else {
                document.getElementById("greeting").innerHTML =
                   "Hello! You did not enter your name.";
            }
        }
    </script>
</head>
<body onload="displayUserName()">

<h2>User Name Display Example</h2>
<p id="greeting"></p>

</body>
</html>
```

Output:

- As soon as the page loads, a **prompt box** appears asking:
  "Please enter your name:"
- If the user enters a name, it is displayed on the page:
  - Example: *Hello, Rajan! Welcome to our website.*
- If the user cancels or leaves it blank, it shows:
  - *Hello! You did not enter your name.*

Observation:

- `prompt()` is used to get input from the user.
- `innerHTML` dynamically updates the content of an HTML element.
- `onload` ensures the prompt appears immediately when the page loads.



Conclusion:

- JavaScript can interactively get input from users and display personalized messages.
- Useful for greetings, forms, and interactive web pages.

Practical: Design HTML Form for Student Record with JavaScript Validation

Aim: To create an HTML form for student records and validate user inputs using JavaScript.

Code:

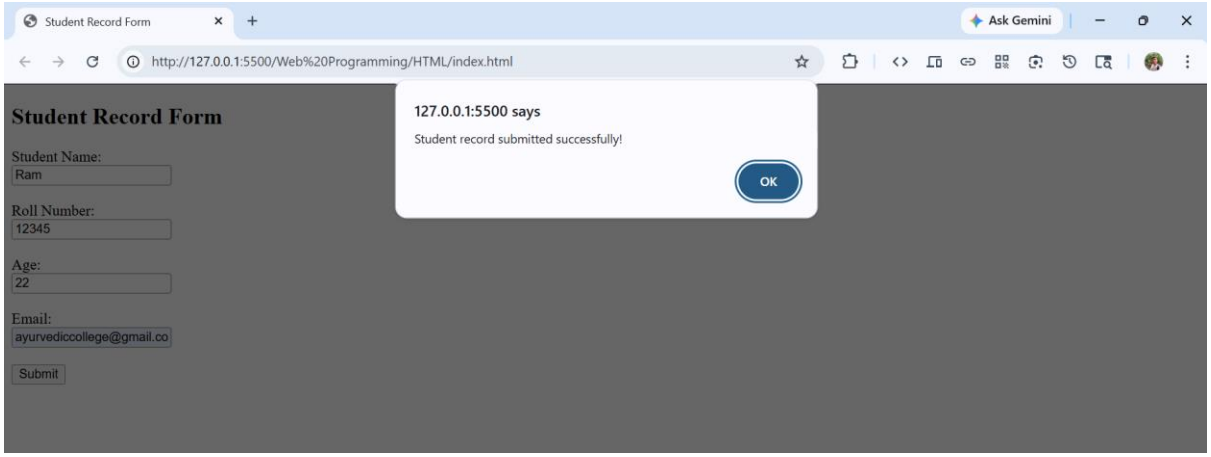
```
<!DOCTYPE html>
<html>
<head>
    <title>Student Record Form</title>
    <script>
        function validateForm() {
            var name = document.forms["studentForm"]["name"].value;
            var roll = document.forms["studentForm"]["roll"].value;
            var age = document.forms["studentForm"]["age"].value;
            var email =
document.forms["studentForm"]["email"].value;

            // Name validation
            if (name == "") {
                alert("Name must be filled out");
                return false;
            }

            // Roll number validation (must be numeric)
```

```
            if (roll == "" || isNaN(roll)) {
                alert("Roll number must be a numeric value");
                return false;
            }

            // Age validation (1-150)
            age = Number(age);
            if (isNaN(age) || age < 1 || age > 150) {
                alert("Age must be a number between 1 and 150");
                return false;
            }

            // Email validation (basic pattern)
            var emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
            if (!email.match(emailPattern)) {
                alert("Please enter a valid email address");
                return false;
            }

            alert("Student record submitted successfully!");
            return true;
        }
    </script>
</head>
<body>

<h2>Student Record Form</h2>

<form name="studentForm" onsubmit="return validateForm()">
    <label for="name">Student Name:</label><br>
    <input type="text" id="name" name="name" placeholder="Enter
student name"><br><br>

    <label for="roll">Roll Number:</label><br>
    <input type="text" id="roll" name="roll" placeholder="Enter roll
number"><br><br>

    <label for="age">Age:</label><br>
    <input type="text" id="age" name="age" placeholder="Enter
age"><br><br>

    <label for="email">Email:</label><br>
```

```
    <input type="text" id="email" name="email" placeholder="Enter
email"><br><br>

    <input type="submit" value="Submit">
</form>

</body>
</html>
```

Output:

- Form with fields: **Student Name**, **Roll Number**, **Age**, and **Email**.
- On clicking **Submit**:
  - **Name** cannot be empty.
  - **Roll Number** must be numeric.
  - **Age** must be between **1 and 150**.
  - **Email** must follow a valid format.
- Alerts appear if any field is invalid.
- If all inputs are valid, alert: **"Student record submitted successfully!"**



Observation:

- JavaScript can validate multiple form fields before submission.
- Alerts help the user correct errors immediately.
- Prevents invalid or incomplete data from being submitted.

Conclusion:

- Form validation ensures **data integrity** and improves **user experience**.
- JavaScript is widely used for client-side validation in web applications.

Practical: Display Browser Information Using JavaScript

Aim: To create a web page that shows information about the browser being used by the user.

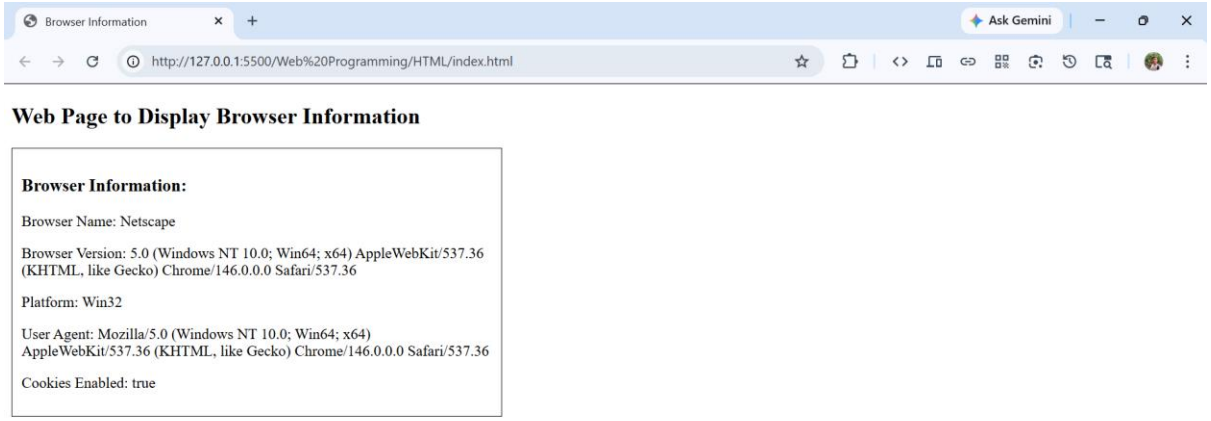
Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Browser Information</title>
    <script>
        function displayBrowserInfo() {
            var info = "<h3>Browser Information:</h3>";
            info += "<p>Browser Name: " + navigator.appName +
"</p>";
            info += "<p>Browser Version: " + navigator.appVersion +
"</p>";
            info += "<p>Platform: " + navigator.platform + "</p>";
            info += "<p>User Agent: " + navigator.userAgent +
"</p>";
            info += "<p>Cookies Enabled: " + navigator.cookieEnabled
+ "</p>";

           // Display information in the div
            document.getElementById("browserInfoDiv").innerHTML =
info;
        }
    </script>
</head>
<body onload="displayBrowserInfo()">

<h2>Web Page to Display Browser Information</h2>

<div id="browserInfoDiv" style="border:1px solid black;
padding:10px; width:500px;">
    <!-- Browser information will appear here -->
</div>

</body>
</html>
```

Output:

- When the page loads, the div displays:

- **Browser Name** (e.g., Netscape)
- **Browser Version** (e.g., 5.0)
- **Platform** (e.g., Win32, MacIntel)
- **User Agent** string
- **Cookies Enabled** status (true/false)



Observation:

- The `navigator` object in JavaScript provides information about the browser.
- `navigator.appName`, `navigator.appVersion`, `navigator.platform`, `navigator.userAgent`, and `navigator.cookieEnabled` are commonly used properties.
- `onload` ensures the information appears automatically when the page loads.

Conclusion:

- JavaScript can access browser properties and provide useful information to the user.
- This can help in browser-specific customizations or debugging web applications.